Computer Architecture Homework 5

Case 1: L1: (8, 8) case

1)Program 1:

No of lines	No of blocks	No of ways	Number of hits	Number of misses	Total number of accesses	Hit rate
1	2	0	49	17	66	0.7424
2	2	0	49	17	66	0.7424
3	2	0	49	17	66	0.7424
3	3	0	57	9	66	0.8636
3	4	0	61	5	66	0.9242
3	5	0	63	3	66	0.9545
3	2	0	49	17	66	0.7424
3	2	1	49	17	66	0.7424

Program 2:

No of lines	No of blocks	No of ways	Number of hits	Number of misses	Total number of accesses	Hit rate
1	2	0	2	64	66	0.0303
2	2	0	2	64	66	0.0303
3	2	0	3	63	66	0.04545
3	3	0	57	9	66	0.8636
3	4	0	61	5	66	0.9242
3	5	0	63	3	66	0.9545
3	2	0	3	63	66	0.04545
3	2	1	49	17	66	0.7424

2)Modified program 1:

Write Policy	Number of hits	Number of misses	Total number of accesses	Hit rate
Write-through (with allocate)	49	17	66	0.7424
Write-through (without allocate)	3	63	66	0.04545
Write-Back (with allocate)	49	17	66	0.7424
Write-Back (without allocate)	3	63	66	0.04545

Modified program 2:

Write Policy	Number of hits	Number of misses	Total number of accesses	Hit rate
Write-through (with allocate)	49	17	66	0.7424
Write-through (without allocate)	3	63	66	0.04545
Write-Back (with allocate)	49	17	66	0.7424
Write-Back	3	63	66	0.04545

(without allocate)				

3)Program 3:

Configuration	Number of hits	Number of misses	Total number of accesses	Hit rate
a)32-entry 4-word direct mapped	2	128	130	0.01538
b)32-entry 4-word 2-way set associative	96	34	130	0.7385
c)32-entry 4-word fully associative	96	34	130	0.7385

Case 2: L1: (16, 16) case

1)Program 1:

No of lines	No of blocks	No of ways	Number of hits	Number of misses	Total number of accesses	Hit rate
1	2	0	193	65	258	0.7481
2	2	0	193	65	258	0.7481
3	2	0	193	65	258	0.7481
3	3	0	225	33	258	0.8721
3	4	0	241	17	258	0.9341
3	5	0	249	9	258	0.9651
3	2	0	193	65	258	0.7481
3	2	1	193	65	258	0.7481

Program 2:

No of lines	No of blocks	No of ways	Number of hits	Number of misses	Total number of accesses	Hit rate
1	2	0	2	256	258	0.007752
2	2	0	2	256	258	0.007752
3	2	0	2	256	258	0.007752
3	3	0	2	256	258	0.007752
3	4	0	3	255	258	0.01163
3	5	0	247	11	258	0.9574
3	2	0	2	256	258	0.007752
3	2	1	2	256	258	0.007752

2)Modified program 1:

Write Policy	Number of hits	Number of misses	Total number of accesses	Hit rate
Write-through (with allocate)	193	65	258	0.7481
Write-through (without allocate)	3	255	258	0.01163
Write-Back (with allocate)	193	65	258	0.7481
Write-Back	3	255	258	0.01163

(without allocate)				

Modified program 2:

Write Policy	Number of hits	Number of misses	Total number of accesses	Hit rate
Write-through (with allocate)	2	256	258	0.007752
Write-through (without allocate)	3	255	258	0.01163
Write-Back (with allocate)	2	256	258	0.007752
Write-Back (without allocate)	3	255	258	0.01163

3)Program 3:

Configuration	Number of hits	Number of misses	Total number of accesses	Hit rate
a)32-entry 4-word direct mapped	35	479	514	0.06809
b)32-entry 4-word 2-way set associative	417	97	514	0.8113
c)32-entry 4-word fully associative	417	97	514	0.8113

Observations:

Program 1:

1)Increasing the number of lines does not improve the hit rate because the amount of memory being accessed sequentially is larger than what the cache can hold and hence every time a line is filled and we go on to a new line, there is a miss. For example, in each line there are 3 hits (for 4 blocks) after the first miss as cache loads all data for the whole line at once.

2)Increasing the number of blocks slightly improves the hit rate because now the cache can hold more amount of data in the same line at once when we start at a new line. So, after hitting a new line, until we reach the end of that line, we will have all hits. For n blocks, we have n – 1 hits, 1 miss, (n – 1)/n = 1 – 1/n which increases as n increases slightly.

3)Increasing the number of ways does not improve the hit rate because the amount of memory being accessed sequentially is larger than what the cache can hold and hence every time a line is filled and we go on to a new line, there is a miss. The number of times we access a new line is based on the number of blocks and not the number of lines.

4)The write policy (write back/write through) does not change the hit rate because write back or write through only changes the way we store modified data in memory and does not change the way we store our data in cache as blocks.

5)Allocation improves the hit rate significantly because no allocation implies the block is not loaded to the cache as we directly write back to memory but with allocation the block is loaded into the

cache, the write is performed and sent back to memory, which has significantly a greater number of hits as multiple words are loaded into memory.

6)As the value of L1 increases, the hit rate improves slightly because it increases the number of cache accesses to adjacent memory locations, which corresponds to an increased number of hits.

Program 2:

1)Increasing the number of lines slightly improves the hit rate because data is now stored in cache at every even index while in the inner loop and then switches to odd indices once the outer loop is reached and this pattern repeats. Now, as the number of lines increases, the number of conflicts decreases, the number of evictions decrease, increasing the number of hits and the hit rate.

2)Increasing the number of blocks slightly improves the hit rate because now more amount of data can be stored in each line, so adjacent data access leads to more number of hits and higher hit rate.

3)Increasing the number of ways improves the hit rate because now more amount of data can be stored in each set, with same index, so adjacent data access leads to more number of hits and higher hit rate.

4)The write policy (write back/write through) does not change the hit rate because write back or write through only changes the way we store modified data in memory and does not change the way we store our data in cache as blocks.

5)Allocation improves the hit rate significantly because no allocation implies the block is not loaded t the cache as we directly write back to memory but with allocation the block is loaded into the cache, the write is performed and sent back to memory, which has significantly a greater number of hits as multiple words are loaded into memory.

6)As the value of L1 increases, the hit rate improves slightly because it increases the number of cache accesses to adjacent memory locations, which corresponds to an increased number of hits.

Program 3:

1)Increasing the number of ways or associativity improves the hit rate because we can store more data at the same index and sometimes the addr1 + 1024 at the start of the loop can match some addr2 in the middle of loop leading to hits.

2)As the value of L1 increases, the hit rate improves because the number of address matches is higher due to higher number of cache accesses.